

N-Body Problem

Scaling Experiments Report

Advanced Programming Techniques

1. Overview

The N-body problem simulates gravitational interactions between N bodies. Each body exerts a force on every other body, making the per-step complexity $O(N^2)$.

Sequential and parallel implementations in Python (multiprocessing) and Rust (std::thread) were benchmarked over 30 independent runs per configuration. Strong scaling used a fixed problem size with increasing core counts; weak scaling grew the problem size proportionally with cores to keep per-core work nominally constant.

Parameter	Value
Steps per run	200
Time step dt	0.001
Strong N (Python)	150
Strong N (Rust)	1000
Weak N ₀ (Python)	50 per core
Weak N ₀ (Rust)	200 per core
Runs per configuration	30
Core counts tested	1, 2, 4, 8, 12

2. Hardware and Software Environment

All experiments were conducted on the same machine throughout. Results may differ on machines with different core counts, cache sizes, or memory bandwidth.

Component	Details
CPU	AMD Ryzen 5 5500U with Radeon Graphics 12-core / 12-thread processor
RAM	System RAM (DDR4 3200 MHz)
OS	Windows 10
Python version	3.12
Rust / Cargo	2021 edition
Python libraries	numpy (inner loop), multiprocessing (stdlib)
Rust crates	rand (body generation), plotters (visualization)
Compiler flags	cargo build --release (full optimization)

3. Serial and Parallel Code Fractions

The serial fraction α is the proportion of the program that cannot be parallelized. It is estimated by fitting Amdahl's law to the measured strong-scaling speedups.

Implementation	Fitted α (serial fraction)	Parallel fraction ($1-\alpha$)	Amdahl max speedup ($p \rightarrow \infty$)
Python (multiprocessing)	0.8495 (84.9%)	15.1%	1.18×
Rust (std::thread)	0.9988 (~100%)	~0%	1.00×

- **Python ($\alpha \approx 0.85$):** The fitted serial fraction of 84.9% does not mean 85% of the algorithm is sequential. The force computation itself is nearly fully parallel. The high α reflects IPC overhead: every simulation step requires pickling all Body objects, sending them to worker processes, and receiving results back. This serialized round-trip dominates runtime at $N=150$ and appears as a large serial fraction to Amdahl's law.
- **Rust ($\alpha \approx 1.00$):** The fitted serial fraction of nearly 100% at $N=1000$ similarly reflects overhead, thread spawning, Arc cloning the body snapshot, per-thread force-array allocation, and the post-join merge loop, those costs more than the parallel computation save at this small problem size. The algorithm itself is highly parallel; the overhead is simply larger than the workload.

4. Strong Scaling (Amdahl's Law)

Strong scaling fixes the problem size and increases the number of parallel workers. Speedup is defined as:

$$S(p) = \frac{T_{\text{sequential}}}{T_{\text{parallel}}(p)}$$

Amdahl's law predicts a theoretical maximum speedup of 1.18×

 for Python and 1.00× for Rust based on the fitted α values. Both are constrained well below ideal linear scaling by overhead costs.

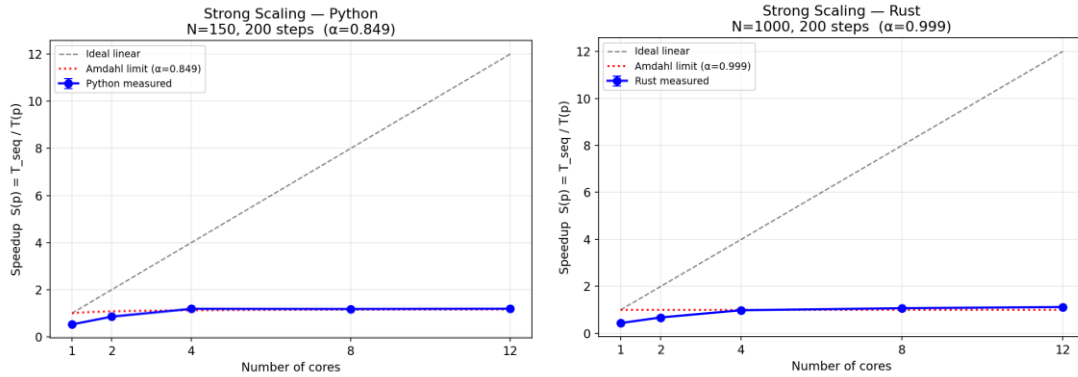


Figure 1. Strong scaling speedup for Python (N=150, left) and Rust (N=1000, right). Error bars show ± 1 standard deviation propagated through the speedup ratio. Dashed black = ideal linear. Red dotted = Amdahl theoretical limit.

4.1 Python Strong Scaling

Sequential baseline: mean = 13.542 s, std = 0.0889 s (30 runs, N=150).

Workers	Runs	Mean time (s)	Std (s)	Speedup S(p)	Speedup $\pm$ err	Outliers
1	30	25.843	0.162	0.524	0.0033	3
2	30	15.843	0.630	0.855	0.0340	8
4	30	11.397	0.310	1.188	0.0323	4
8	30	11.470	0.172	1.181	0.0177	1
12	30	11.351	0.082	1.193	0.0087	3

Parallel with 1 worker is slower than sequential ($S=0.52$) because pickle serialization of 150 Body objects per step adds overhead exceeding any computation benefit. Speedup plateaus sharply after 4 workers, at N=150 the parallelizable work is exhausted and overhead dominates. The maximum achieved speedup is 1.19 $\times$ at 4 workers, essentially unchanged at 8 and 12.

4.2 Rust Strong Scaling

Sequential baseline: mean = 0.6005 s, std = 0.00863 s (30 runs, N=1000).

Threads	Runs	Mean time (s)	Std (s)	Speedup S(p)	Speedup $\pm$ err	Outliers
1	30	1.3657	0.01914	0.440	0.0062	1
2	30	0.8854	0.03962	0.678	0.0303	—
4	30	0.6096	0.01630	0.985	0.0263	—
8	30	0.5603	0.01616	1.072	0.0309	—
12	30	0.5350	0.01614	1.122	0.0339	3

The pattern mirrors Python but with far smaller absolute times. Parallel-1-thread costs 1.37s vs sequential 0.60s, thread spawning, Arc clone, and merge loop add ~0.77s of overhead per call. Speedup improves steadily to 1.12× at 12 threads. The absence of meaningful speedup confirms N=1000 is too small for Rust parallelism to pay off; overhead is comparable to the computation itself.

5. Weak Scaling (Gustafson's Law)

Weak scaling grows the problem size in proportion to the number of cores, keeping per-core work nominally constant. Efficiency is:

$$E(p) = \frac{T(1, N_0)}{T(p, p * N_0)}$$

Gustafson's law predicts the scaled speedup $S_G(p) = p - \alpha * (p-1)$, or equivalently efficiency $E(p) = (p - \alpha(p-1))/p$.

Implementation	Gustafson α	$E(p)$ at p=12 (theory)	$E(p)$ at p=12 (measured)
Python	0.9990	0.084	0.032
Rust	0.9990	0.084	0.0485

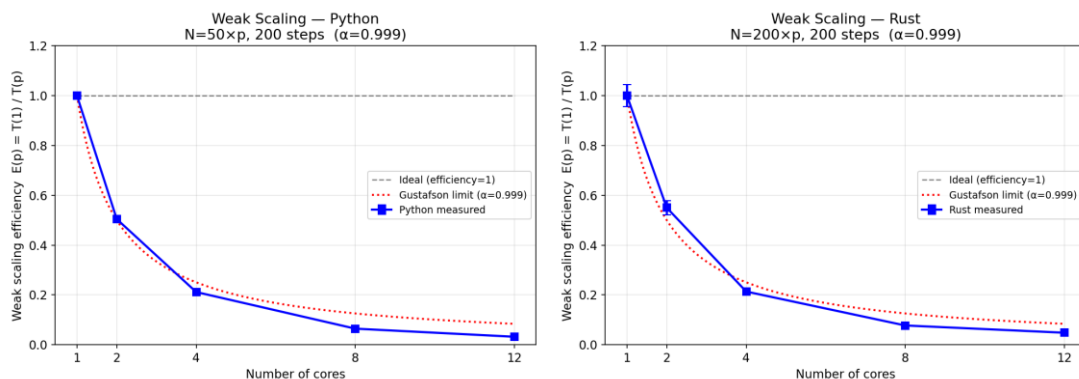


Figure 2. Weak scaling efficiency for Python (N=50×p, left) and Rust (N=200×p, right). Error bars show ±1 std propagated through the efficiency ratio.

5.1 Python Weak Scaling

N per core = 50. Baseline: N=50, 1 worker, mean = 4.013 s.

Workers	N	Runs	Mean (s)	Std (s)	Efficiency	Eff ±err	Outliers
1	50	30	4.013	0.059	1.000	0.0146	1
2	100	30	7.953	0.085	0.505	0.0054	4
4	200	30	18.952	0.499	0.212	0.0056	4
8	400	30	61.936	0.715	0.065	0.0007	2
12	600	30	124.185	4.150	0.032	0.0011	5

Efficiency collapses from 100% at $p=1$ to 50.5% at $p=2$, 21.2% at $p=4$, 6.5% at $p=8$, and 3.2% at $p=12$. This is steeper than the $1/p$ theoretical $O(N^2)$ collapse because pickle serialization cost also grows with N .

5.2 Rust Weak Scaling

N per core = 200. Baseline: $N=200$, 1 thread, mean = 0.0980 s.

Threads	N	Runs	Mean (s)	Std (s)	Efficiency	Eff $\pm$ err	Outliers
1	200	30	0.0980	0.00431	1.0000	0.04391	1
2	400	30	0.1782	0.00901	0.5502	0.02783	2
4	800	30	0.4587	0.01969	0.2137	0.00917	2
8	1600	30	1.2640	0.05812	0.0776	0.00357	3
12	2400	30	2.0235	0.04230	0.0485	0.00101	5

Efficiency follows the same $1/p$ collapse. This collapse is a mathematical consequence of the $O(N^2)$ complexity: doubling N quadruples work, so doubling cores and N simultaneously doubles each core's load. For Gustafson's law to hold exactly, N would need to scale as $\sqrt{p}$ rather than p .

6. Simulation Visualization

Figure 3 shows body trajectories from the Rust parallel simulation ($N=2400$, 5000 steps). The spiral arms and scattered paths confirm that gravitational perturbations between satellite bodies break initial circular orbits, producing expected chaotic many-body dynamics. The central massive body (magenta dot) remains near the origin.

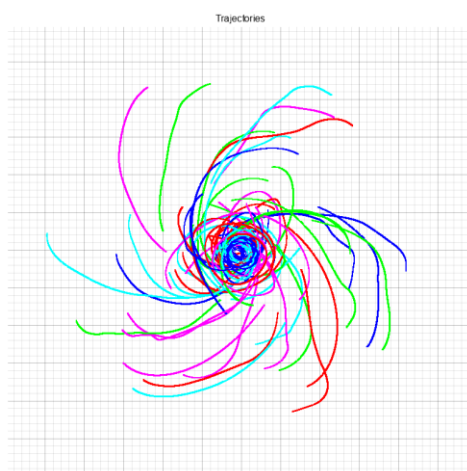


Figure 3. Body trajectories – Rust parallel, $N=2400$, 5000 steps. Each color represents one body. The central massive body remains near the origin.

7. Summary of Results

Metric	Python	Rust
Sequential baseline	13.542 s (\$N=150\$)	0.600 s (\$N=1000\$)
Strong scaling max speedup	1.19× at 4 workers	1.12× at 12 threads
Par-1-worker overhead	~91% slower than seq	~127% slower than seq
Fitted Amdahl α	0.8495 (overhead-dominated)	0.9988 (overhead-dominated)
Amdahl theoretical max	1.18×	1.00×
Weak scaling E(p=12)	3.2%	4.9%

Both strong and weak scaling are dominated by overhead rather than algorithmic serial fraction. For strong scaling, the tested problem sizes are too small for parallel computation to exceed parallelization overhead. For weak scaling, the $O(N^2)$ complexity causes efficiency to collapse as $1/p$ regardless of language, which is the mathematically expected outcome when N grows linearly with p for a quadratic algorithm.